

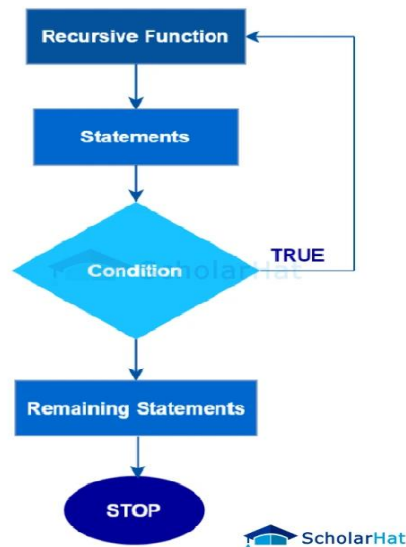
Recursion

In general terms recursion means the process to define a problem or the solution for a problem in a much simpler way compared to the original version. It is a problem-solving programming technique that has a remarkable and unique characteristic.

Recursion in data structure is additionally an adequate programming technique. A recursive subroutine is described as one that directly or indirectly calls itself. Calling a subroutine specifically indicates that the characterization of the subroutine already has the call statement of calling the subroutine that has been defined.

A function that is defined in terms of itself is called recursive. The process of recursion involves solving a problem by turning it into smaller varieties of itself. Recursion in the data structure is one of the most compact and improved strategies for creating function calls or deploying functions in programming. It helps you to implement many functions and algorithms efficiently, at the same time

For example :- folder could be a storage device that is used to store files and folders. defining a folder could be a tough task. It could be anything that holds some files and folders that could be files and folders in their own right, and this could again go on. This is why recursion makes defining situations a lot easier than usual.



A recursive function is defined in terms of base cases and recursive steps.

- In a **base case**, we compute the result immediately given the inputs to the function call.
- In a recursive step, we compute the result with the help of one or more recursive calls to this same function, but with the inputs somehow reduced in size or complexity, closer to a base case.
- If **base case** is not define” it will raise an error as “Internal Stack Overflow”.
- A recursive algorithm should change its move and mention the base case.
- The **recursive case** defines how the function breaks down a larger problem into smaller sub-problems and utilizes recursive calls to solve them.
- A **partial solution** refers to the progress made towards the final solution during a recursive call.

Advantages of Recursion

1. Sometimes, in programming, a problem can be solved without recursion, but at some situations in programming, it is must to use recursion. For example, a program to display the list of all files of the system cannot be solved without recursion.

2. The recursion is very flexible in data structure like stacks, queues, linked list and quick sort.
3. Using recursion, the length of the program can be reduced.

Disadvantages of Recursion

- Recursion consumes more storage space because the recursive calls along with automatic variables are stored on the stack.
- The computer may run out of memory if the recursive calls are not checked.
- It is not more efficient in terms of speed and execution time.
- According to some computer professionals, recursion does not offer any concrete advantage over non-recursive procedures/functions.
- If proper precautions are not taken, recursion may result in non-terminating iterations.
- Recursion is not advocated when the problem can be through iteration. Recursion may be treated as a software tool to be applied carefully and selectively.

Application of Recursion

- **Solving Problems with Recursive Structure:** Recursion is particularly useful for solving problems that exhibit recursive structure, such as tree traversal, graph traversal, and divide-and-conquer algorithms like quicksort and mergesort.
- **Mathematical Calculations:** Recursion is often used in mathematical calculations that can be defined recursively, such as calculating factorials, Fibonacci numbers, and the Tower of Hanoi problem.
- **Data Structures:** Recursion is commonly used in data structures like trees and graphs, where a recursive approach can simplify operations like tree traversal, searching, and manipulation.
- **Dynamic Programming:** Many dynamic programming problems can be solved using recursion. In dynamic programming, subproblems are solved recursively and their solutions are stored in a table to avoid recomputation.
- **Backtracking:** Backtracking algorithms often use recursion to explore all possible solutions to a problem by trying out different choices at each step and backtracking when a solution is not found.
- **Functional Programming:** Recursion is a fundamental concept in functional programming languages like Lisp, Scheme, and Haskell, where it is used extensively due to the lack of mutable state and loops.
- **Algorithm Design:** Recursion is used in designing algorithms for problems where a problem can be broken down into smaller subproblems of the same type.

Recursion Algorithm

A recursive algorithm calls itself with small input values & returns the outcome for the present input by executing fundamental functions on the returned value for the small input. By and large, if a problem could be sorted out by using remedies to smaller adaptations of the same problem, in addition to the smaller variations reduced to conveniently solvable cases, then the problem could be fixed employing a recursive algorithm.

1. Greatest Common Divisor
2. Sum of Natural Numbers
3. Factorial of a Positive Integer
4. Fibonacci Series
5. Tower of Hanoi

1. Greatest Common Divisor

The Greatest Common Divisor (GCD) of two integers is the largest positive integer that divides both numbers without leaving a remainder. GCD is commonly used in many mathematical computations and algorithms.

Let's take an example to find the GCD of two numbers, say 48 and 18.

Finding GCD:

14) 48 (3

42

6

Remainder is not yet zero, so we will now divide 14 by 6

14) 48 (3

42

6) 14 (2

12

2

Remainder is not yet zero, so we will now divide 6 by 2

14) 48 (3

42

6) 14 (2

12

2) 6 (3

6

0

Remainder is 0 so, we stop here

Therefore, the required GCD is 2

//Program will be provided in Teams

2. Sum of Natural Number

The positive numbers 1, 2, 3... are known as natural numbers. The program below takes a positive integer from the user and calculates the sum up to the given number.

5.4 Factorial of a number

Following program demonstrate the recursive method for finding a factorial of a number.

```
#include<stdio.h>
int Factorial(int number)
// Pre: number is nonnegative.
// Post: Function value = factorial of number.
{
    if (number == 0) // Line 1
        return 1; // Line 2
    else
        return number * Factorial(number-1); // Line 3
}
int main()
{
    int num,fact=0;
    //Initialization of formal parameters, local variables and etc.
    printf("\n Enter Number ");
    scanf("%d",&num);
    //Starting point of the function, which calls itself
    fact = Factorial(num);
    printf("Factorial of %d is %d \n",num,fact);
    return 0;
}
```

Notice the use of Factorial in **line 3**. Factorial involves a recursive call to the function, with the parameter *number*. Let's walk through the calculation of 4! using the function Factorial. The original value of number is 4. Following Table shows the steps in the calculation.

Recursive Call	Line	Action
	1 3	4 is not 0, so skip to else clause. 3 return <i>number * Factorial (4 - 1)</i> First recursive call returns us to the beginning of the function with <i>number = 3</i> .
1	1 3	3 is not 0, so skip to else clause. 3 return <i>number * Factorial (3 - 1)</i> Second recursive call returns us to the beginning of the function with <i>number = 2</i> .
2	1 3	2 is not 0, so skip to else clause. 3 return <i>number * Factorial (2 - 1)</i> Third recursive call returns us to the beginning of the function with <i>number = 1</i> .
3	1 3	1 is not 0, so skip to else clause. 3 return <i>number * Factorial (1 - 1)</i> Fourth recursive call returns us to the beginning of the function with <i>number = 0</i> .
4	1 2 3	0 is 0, so go to line 2. return 1 1 replaces call to <i>Factorial</i> ; <i>number</i> is 1. return 1
3	3	1 replaces call to <i>Factorial</i> ; <i>number</i> is 2. return 2
2	3	2 replaces call to <i>Factorial</i> ; <i>number</i> is 3. return 6
1	3	replaces call to <i>Factorial</i> ; <i>number</i> is 4. return 24

Table 3 : Recursive call of 4!

5.5 Tower of Hanoi

Let us look at the Tower of Hanoi problem and see how we can use recursive technique to produce a logical and elegant solution. Let us suppose that we have three towers A, B, C and three different sizes disks let say 1, 2 and 3. Each disk has a hole in the center so that it can be stacked on any of the pegs. At the beginning, the disks are stacked on the A peg, that is the largest sized disk on the bottom and the smallest sized disk on top. Here we have to transfer all the disks from source peg A to the destination peg C by using an intermediate peg B. Following are the rules to be followed during transfer:

1. Transferring the disks from the source peg to the destination peg such that at any point of transformation no large size disk is placed on the smaller one.
2. Only one disk may be moved at a time.
3. Each disk must be stacked on any one of the pegs.

Now following could be the steps to solve the tower of Hanoi problem for 3 disks.

```
Move disc 1 from A to C
Move disc 2 from A to B
Move disc 1 from C to B
Move disc 3 from A to C
Move disc 1 from B to A
Move disc 2 from B to C
Move disc 1 from A to C
```

We can generalize the solution to the tower of Hanoi problem recursively as follows:

To move n disks from peg A to peg C, Using B as auxiliary peg.

1. If $n = 1$, move the single disk from A to C and stop.
2. Move the $top(n - 1)$ disks from the peg A to the peg B, using C as auxiliary.
3. Move n th disk to peg C.
4. Now move $n - 1$ disk from B to C, using A as auxiliary.

Below Program simulate the tower of Hanoi problem.

```
#include<stdio.h>
void tower_of_hanoi(int n, char from, char temp, char to)
{
    if (n == 0)
        return;
    tower_of_hanoi(n-1, from, to, temp);
    printf("Move disc %d",n);
    printf(" from %c to %c \n",from,to);
    tower_of_hanoi(n-1, temp, from, to);
}
int main()
{
    int n;
    printf("Enter the number of disks : ");
    scanf("%d",&n);
    tower_of_hanoi(n, 'A', 'B', 'C');
    return 0;
}
```

}

Towers of Hanoi: Recursion Tree

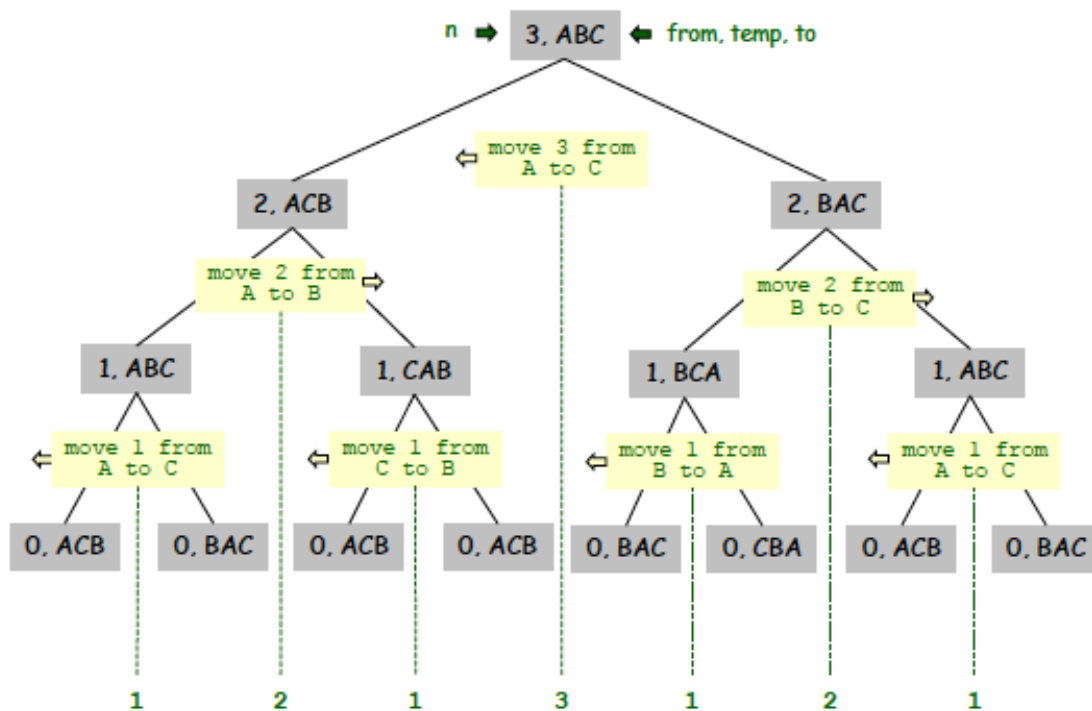


Figure 16 : Recursive Tree of TOH

5.6 Fibonacci Series

Fibonacci can be defined recursively as follows. Each number is the sum of the previous two numbers in the sequence. The n^{th} Fibonacci number is

$$\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$$

$$\text{Fib}(0) = 1,$$

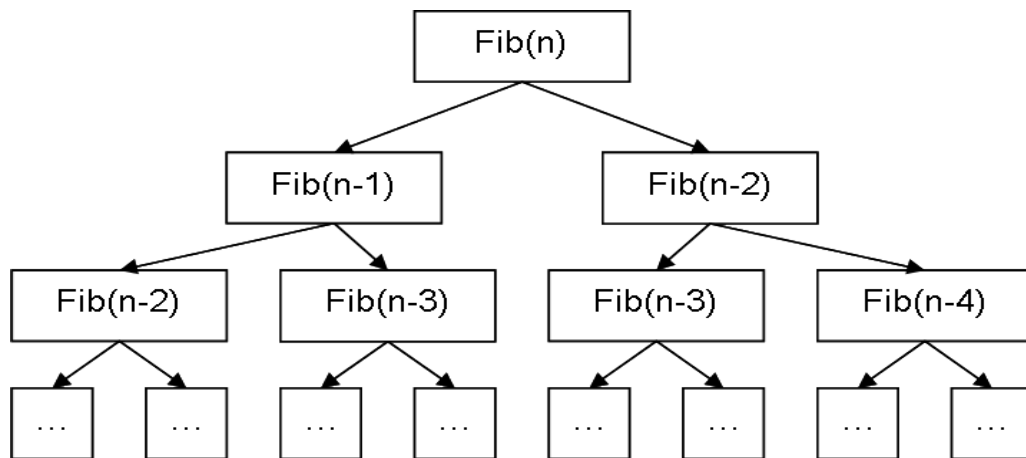
$$\text{Fib}(1) = 1.$$

That is, the n^{th} number is formed by adding the two previous numbers together. The equation for $\text{Fib}(n)$ is said to be the recurrence relation and the terms for $\text{Fib}(0)$ and $\text{Fib}(1)$ are the base cases. The code for the recursive Fibonacci sequence is provided in below.

```
int Fib (int n) {
    if (n<=1)
        return 0;
    else
        return ( Fib(n-1) + Fib(n-2) );
}
```


To find $Fib(n)$ we need to call Fib two more times: once for $Fib(n-1)$ and once more for $Fib(n-2)$; then we add the results. But to find $Fib(n-1)$, we need to call Fib two more times: once for $Fib(n-2)$ and once more for $Fib(n-3)$; then we need to add the results. And so on! Below is the recursion tree for $Fib(n)$ in Figure. The recursion tree shows each call to $Fib()$ with different values for n .

Figure : The recursion tree for $Fib(n)$. <http://www.soe.ucsc.edu/~fire/dev-2008f-12/labs/lab8-Recursion-vs-Iteration/fib1.png>



[12/labs/lab8-Recursion-vs-Iteration/fib1.png](http://www.soe.ucsc.edu/~fire/dev-2008f-12/labs/lab8-Recursion-vs-Iteration/fib1.png)

For example, consider $Fib(5)$, the fifth term in the sequence. To find $Fib(5)$, we have to find $Fib(4) + Fib(3)$. But to find $Fib(4)$, it is needed to find $Fib(3) + Fib(2)$. Luckily, $Fib(2)$ is a base case; the answer to $Fib(2)$ is 1. And so on. Below figure is the example recursion tree for $Fib(5)$; it lists all of the computations needed to carry out the calculation recursively.

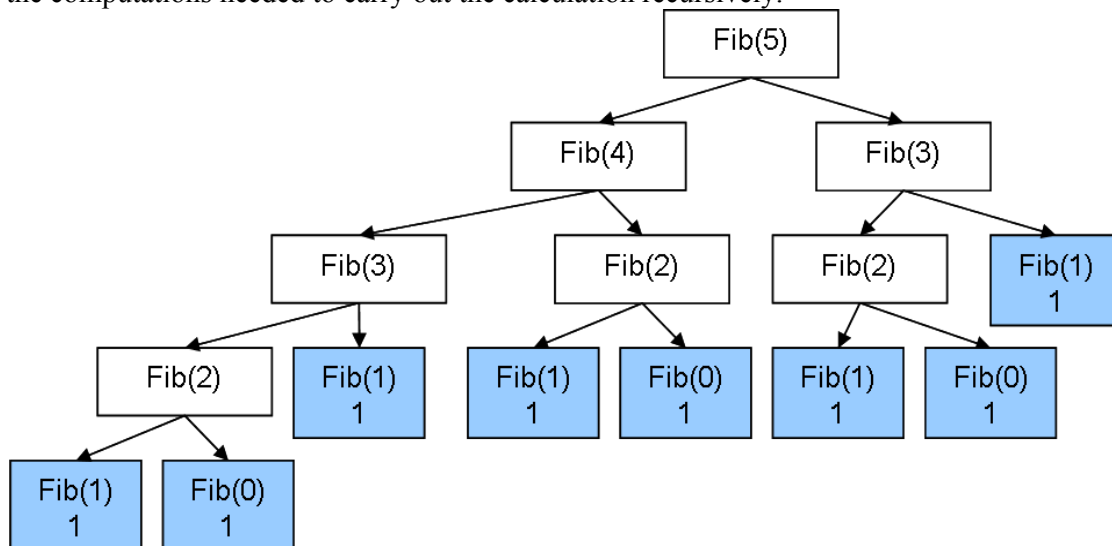


Figure 17: The recursion tree for $Fib(5)$, Base cases are highlighted

This routine works but has a serious problem. It performs quite badly. (*fib(40)* takes 4 minutes to compute). **Problem:** Redundant calculations. To compute $fib(n)$ we recursively compute $fib(n-1)$, when this call returns we compute $fib(n-2)$. But we have already computed $f(n-2)$ in the process of computing $f(n-1)$, so the call to $fib(n-2)$ is wasted. It is a redundant calculation. Note that: The redundancy increases for each recursive call.

Program to find sum of first n natural numbers

$$6 \\ 6+5+4+3+2+1 = 21$$

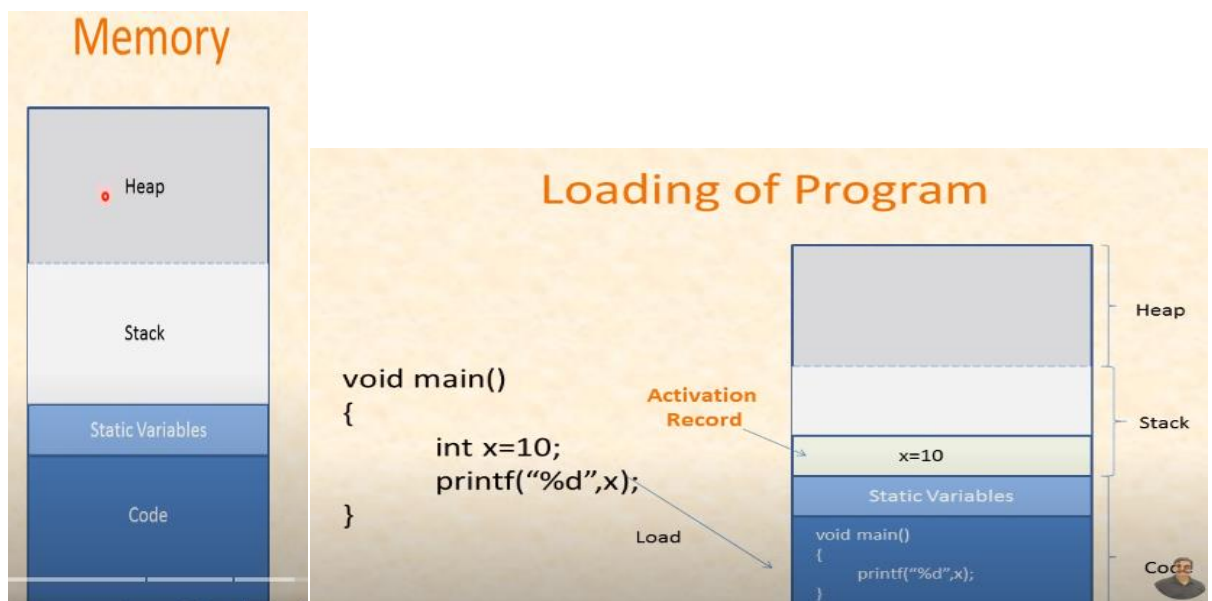


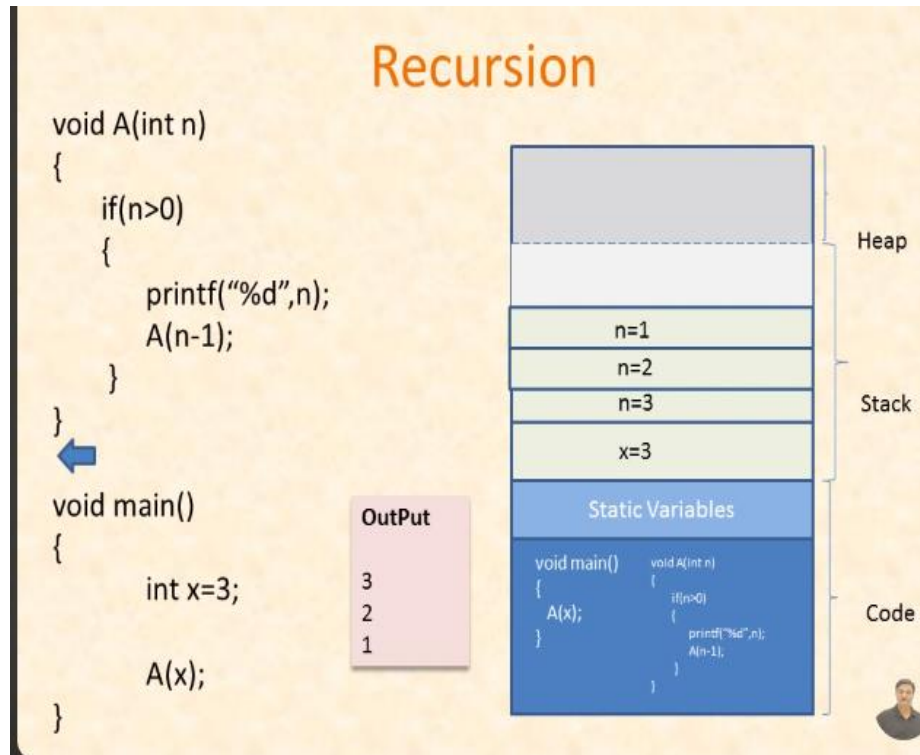
Recursion and Stack

Recursion and stacks are closely related concepts in computer science and play a fundamental role in data structures and algorithms (DSA)

How Stack Works with Recursion:

- **Function Calls:** When a function is called recursively, information about each function call, such as parameters and local variables, is pushed onto the call stack. When a function is called, an activation record is created and pushed onto the call stack.
- **Memory Allocation:** Each recursive call consumes additional memory on the stack. If the recursion depth is too deep, it may lead to a stack overflow error.
- **Return Values:** When a base case is reached, the recursion stops, and the function calls begin to return, its values are propagated back through the call stack, allowing the program to compute the final result.
- **Stack Frames:** Each function call creates a stack frame (activation record), which contains information about the function's execution context. When a function returns, its stack frame is removed from the call stack, freeing up memory.





Recursion Vs Iteration

Recursion involves a function calling itself to solve a problem, breaking it down into simpler subproblems. It's elegant and suitable for problems with unknown or varying iterations, but may suffer from performance overhead and stack overflow issues.

Iteration, on the other hand, uses loops to repeat a set of instructions until a condition is met. It's efficient and predictable, making it suitable for problems with a fixed number of iterations. However, it can result in complex code and is less suitable for certain types of problems that are naturally recursive.

RECURSION	ITERATION
Recursion function calls itself	Iteration function are repeatedly executed using conditional loop
Recursion function is terminated when the base case is reached.	Iteration function is terminated when the loop condition fails.
Recursion uses Stack memory to store the current state of the function.	Iteration function does not use stack memory.
Recursion function takes up more memory	Iteration function takes up less memory.
Infinite recursion function occurs if the recursion step does not reduce the problem in a manner that converges on the base case.	Infinite iteration function occurs if the loop condition never becomes false.
The execution of recursion is comparatively slower.	The execution of iteration is comparatively faster

Recursive Data Structure

Recursive data structures are data structures that contain references to instances of the same type within their own definition. This self-referential characteristic allows for natural and efficient recursive algorithms to be applied to operations on these data structures. Examples include linked lists, trees, and graphs. Recursive data structures facilitate elegant and concise implementations of algorithms due to their inherent recursive nature, making them fundamental in many areas of computer science and programming.

Recursion and Abstract Data Types (ADTs) are related in how recursion can be used to implement operations defined by ADTs efficiently. Recursive algorithms are commonly employed for operations like traversal, search, insertion, and deletion in data structures such as trees and linked lists. Sometimes, ADT designs explicitly incorporate recursion to define their behaviors, especially for data structures with recursive properties like trees and graphs. Thus, recursion plays a significant role in solving problems related to ADTs and in designing efficient data structures.

Types of Recursion

5.2 Types of Recursion

Recursion is a technique that breaks a problem into one or more sub-problems that are similar to the original problem. Any recursive function can be characterized based on:

- whether the function calls itself directly or indirectly (direct or indirect recursion),
- whether any operation is pending at each recursive call (tail recursive or not), and
- the structure of the calling pattern (linear or tree-recursive).

```
int Func (int n)
{
    if (n == 0)
        return n;
    else
        return (Func (n-1));
}
```

Direct Recursion

```
int Func1 (int n)
{
    if (n == 0)
        return n;
    else
        return Func2(n);
}
int Func2(int x)
{
    return Func1(x-1);
}
```

Indirect Recursion

```
int Fact(int n)
{
    if (n == 1)
        return 1;
    else
        return (n * Fact(n-1));
}
```

Non-tail recursion

```
int Fact(n)
{
    return Fact1(n, 1);
}
int Fact1(int n, int res)
{
    if (n == 1)
        return res;
    else
        return Fact1(n-1, n*res);
}
```

Tail recursion

- In simple words, a recursive function is said to be linearly recursive when the pending operation (if any) does not make another recursive call to the function. For example, observe the last line of recursive factorial function. The factorial function is linearly recursive as the pending operation involves only multiplication to be performed and does not involve another recursive call to Fact. On the contrary, a recursive function is said to be tree recursive (or non-linearly recursive) if the pending operation makes another recursive call to the function. For example, the Fibonacci function in which the pending operations recursively call the Fibonacci function.

```
int Fibonacci(int num)
{
    if(num == 0)
        return 0;
    else if (num == 1)
        return 1;
    else
        return (Fibonacci(num - 1) + Fibonacci(num - 2));
}
```

Observe the series of function calls. When the function returns, the pending operations in turn calls the function

Fibonacci(7) = Fibonacci(6) + Fibonacci(5)

Fibonacci(6) = Fibonacci(5) + Fibonacci(4)

Fibonacci(5) = Fibonacci(4) + Fibonacci(3)

Fibonacci(4) = Fibonacci(3) + Fibonacci(2)

Fibonacci(3) = Fibonacci(2) + Fibonacci(1)

Fibonacci(2) = Fibonacci(1) + Fibonacci(0)

Now we have, Fibonacci(2) = 1 + 0 = 1

Fibonacci(3) = 1 + 1 = 2

Fibonacci(4) = 2 + 1 = 3

Fibonacci(5) = 3 + 2 = 5

Fibonacci(6) = 5 + 3 = 8

Fibonacci(7) = 8 + 5 = 13

Tree recursion